Project Report: Fine-Tuning a Large Language Model (LLM) for Text Classification

Project Focus: Large Language Models (LLM), Transfer Learning, NLP Pipelines

1. Executive Summary

This document outlines the methodology, execution, and results of fine-tuning a pre-trained Large Language Model (LLM) for a specific Natural Language Processing (NLP) task. The project successfully utilized Transfer Learning to adapt the distilbert-base-uncased model to perform binary sentiment analysis (Positive/Negative) on the IMDb Movie Review Dataset. After two epochs of fine-tuning utilizing GPU acceleration, the model achieved a final Accuracy of 92.32% and an F1-Score of 92.14%.

2. Technical Environment & Prerequisites

To ensure reproducibility and efficient processing, a dedicated virtual environment was constructed utilizing Anaconda. The pipeline heavily relied on PyTorch for tensor operations and Hugging Face ecosystem libraries for model architecture and data handling.

Core Dependencies:

- **Python:** 3.10
- **Deep Learning Framework:** PyTorch 2.5.1 (compiled with NVIDIA CUDA 11.8 for GPU acceleration)
- **NLP Libraries:** Hugging Face transformers, datasets, and evaluate
- **Computation & Metrics:** scikit-learn, numpy

Hardware acceleration was achieved by routing the Hugging Face Trainer API to utilize local NVIDIA GPU resources via the accelerate library, drastically reducing training time from hours (CPU-bound) to minutes.

3. Data Acquisition and Preprocessing

The model was trained and evaluated using the public **IMDb Movie Review Dataset**, which consists of 25,000 highly polar movie reviews for training and 25,000 for testing.

3.1 Tokenization

LLMs cannot process raw text; they require numerical representations. The AutoTokenizer corresponding to the distilbert-base-uncased checkpoint was loaded to handle this translation.

A preprocessing function was mapped across the entire dataset to apply tokenization. Key parameters included:

- **Truncation (truncation=True):** Ensuring no review exceeded the maximum sequence length permitted by DistilBERT (512 tokens).
- **Padding (padding="max_length"):** Standardizing the length of all input vectors by padding shorter reviews, ensuring uniform tensor shapes for batch processing.

4. Model Architecture & Fine-Tuning Methodology

The project utilized **DistilBERT**, a smaller, faster, and lighter version of BERT that retains 97% of its language understanding capabilities while being 60% faster.

4.1 Transfer Learning Application

Rather than initializing a neural network with random weights and training it from scratch on 25,000 movie reviews (which would yield poor results and require massive computing power), Transfer Learning was employed. The distilbert-base-uncased model has already learned grammar, syntax, and complex language patterns from reading vast amounts of the English Wikipedia and the Toronto Book Corpus.

During this project, we initialized this pre-trained model and replaced its final layer with a new classification head initialized for 2 labels (Positive and Negative). During fine-tuning, the model simply adjusted its existing knowledge to fit the specific nuance of movie review sentiment.

4.2 Training Hyperparameters

The Hugging Face Trainer API was utilized to handle the training loop with the following TrainingArguments:

- **Epochs:** 2
- **Learning Rate:** 2e-5 (A standard, low learning rate to prevent catastrophic forgetting of the pre-trained weights).
- **Batch Size:** 16 (per device, for both training and evaluation).
- **Weight Decay:** 0.01 (To prevent overfitting).
- **Evaluation Strategy:** Epoch-based. The model calculated validation metrics at the end of each epoch and was instructed to load the best-performing model weights at the end of the run (load_best_model_at_end=True).

5. Evaluation & Results

The model's performance was evaluated against the unseen 25,000-sample test dataset using Accuracy and F1-Score metrics.

Final Evaluation Metrics:

- **Evaluation Loss:** 0.1977
- **Accuracy:** 0.9232 (92.32%)
- **F1-Score:** 0.9214 (92.14%)

The near-identical Accuracy and F1-Score indicate a highly balanced model that performs equally well at identifying both positive and negative sentiments without bias toward one class. The model effectively identified complex sentiment markers within the text.

6. Real-World Inference

To validate the model's practical utility, a custom inference function was developed. This function tokenizes raw input strings, passes them through the fine-tuned model without calculating gradients (torch.no_grad()), and extracts the highest probability class logit.

Test Case 1:

- *Input:* "The pacing was a bit slow, but the cinematography was absolutely breathtaking!"
- *Output:* Positive

Test Case 2:

- *Input:* "I honestly couldn't wait for this movie to end. Terrible acting."
- *Output:* Negative

7. Conclusion: Power and Limitations of Pre-Trained LLMs

The fine-tuning process was highly successful. Achieving over 92% accuracy after only two epochs demonstrates the immense power of Transfer Learning.

Strengths:

Pre-trained LLMs provide excellent out-of-the-box generalization. They require significantly smaller task-specific datasets to achieve high accuracy and save immense amounts of computational resources and engineering time.

Limitations:

Despite their power, LLMs can be large and memory-intensive to run in production environments. Furthermore, because they are trained on vast, uncurated internet text, they can inherit and amplify societal biases present in their original training data. Finally, they operate as "black boxes"—while we know *what* the model predicted, it remains mathematically difficult to definitively explain exactly *why* it made a specific classification.